Gap 1 — Hardcoded secrets in the AI system prompt (F-03) · Critical · Easy

Technical finding: the TechCorp assistant's system prompt contains a live database password and an API key (`sk-techcorp-9f3a1c7e2b`), and the MCP capability-chaining attack in Project 4 extracted the same API key via `list_dir → read_file → query_db`. Any user who can chat with the assistant, or reach the tool server, can retrieve credentials. Regulatory obligations violated: ASVS V6.4.1 requires secrets to live in a managed vault, never in source or configuration; NIST CSF PR.AC-1 requires identities and credentials to be managed and protected; EU AI Act Article 10 (data governance) and Article 15 (cybersecurity) require that a high-risk AI system protect the data and credentials it processes. Business risk: the exposed credentials grant direct access to the backing database and third-party API, so a single prompt can escalate to full data exfiltration and lateral movement; because the secrets are already disclosed they must be treated as compromised and rotated regardless of other fixes, and any personal data reached through them triggers breach-notification duties.

Gap 2 — SQL injection and authentication bypass (F-01, F-02) · Critical · Medium

Technical finding: the DVWA User-ID parameter concatenates input into a SQL query, so `1' OR '1'='1` dumped all five users and `' UNION SELECT user,password FROM users#` extracted every MD5 password hash; the admin hash `5f4dcc3b5aa765d61d8327deb882cf99` trivially cracks to `password`, yielding admin takeover. Regulatory obligations violated: ASVS V5.3.4 (parameterised queries) and V2.4.1 (approved one-way password hashing — MD5 is prohibited); NIST CSF PR.DS-5 (protect against data leaks) and PR.AC-1 (credential management); EU AI Act Article 15 (accuracy, robustness and cybersecurity). Business risk: an unauthenticated attacker can read the entire user store and authenticate as any user, including administrators, leading to complete confidentiality and integrity loss of the application and its data — the highest-impact web finding in the engagement.

Gap 3 — MCP arbitrary command execution and path traversal (F-08, F-09) · Critical · Hard

Technical finding: the MCP tool server exposes `execute_command` (confirmed RCE — `whoami` returned `nayaa`) and `read_file` with no path validation (confirmed arbitrary read of `C:\Windows\win.ini` and `../../../../etc/passwd`). These tools let a model — or anyone who can drive it — run host commands and read any file. Regulatory obligations violated: ASVS V5.3.8 (prevent OS command injection; no `shell=True`) and V12.3.1 (validate user-supplied paths / prevent traversal); NIST CSF PR.AC-4 (least privilege — the tools grant far more than the assistant needs) and PR.DS-5 (data-leak protection); EU AI Act Article 15 (robustness and cybersecurity of a high-risk system). Business risk: command execution on the host is the most severe technical outcome possible — full server compromise, data theft, and a pivot point into TechCorp's network — and because it stems from the tool architecture it is the hardest to fully remediate, requiring capability redesign rather than a one-line fix.

Gap 4 — CI/CD pipeline without security gates and vulnerable dependencies (F-11, F-12) · High · Medium

Technical finding: the deployment pipeline shipped code with no automated security checks; Project 5 proved that a fail-state commit carrying hardcoded secrets, an `eval()` call, an injection-phrase system prompt and 63 known dependency CVEs across six packages would have reached production, and only the gates built in that project (Bandit, pip-audit/Safety, Gitleaks, custom scanner) stop it. Regulatory obligations violated: ASVS V14.1.1 (build pipeline must enforce SAST/secret/dependency scanning) and V14.2.1 (components free of known vulnerabilities); NIST CSF PR.IP-2 (a secure SDLC) and ID.RA-1 (vulnerabilities identified and documented); EU AI Act Article 9 (a risk management system operating across the lifecycle). Business risk: without gates, every one of the other findings can silently regress into production after remediation, so this gap undermines the durability of the entire remediation programme; unpatched dependency CVEs are also directly exploitable by anyone.

Gap 5 — Prompt injection and missing input validation (F-04, F-10) · High · Medium

Technical finding: the Flask AI app performs no input validation and no output scanning, so direct prompt injection (Project 1/3) and indirect/resource-poisoning injection through MCP tool content (Project 4) can steer the assistant to disclose its system prompt or invoke privileged tools. The local `llama3.2:3b` model happened to refuse most direct attempts, but model alignment is not a security control. Regulatory obligations violated: ASVS V5.1.3 (validate and sanitise all untrusted input); NIST CSF PR.DS-5 (data-leak protection) and PR.AC-4 (least-privilege tool authorization); EU AI Act Article 15 (robustness against manipulation). Business risk: a reliable injection lets an attacker bypass the assistant's instructions, exfiltrate the embedded secrets (Gap 1) and chain into the MCP tools (Gap 3), turning the chat box into an entry point for the most severe findings; the defence must be structural (validation + output scanning + per-tool authorization), independent of which model is deployed.